

# Streamlit with Python

## 1. Introduction to Streamlit

### What is Streamlit?

Streamlit is an open-source Python framework used to build interactive web applications, dashboards, data science applications, machine learning applications, and AI applications with minimal code.

Unlike traditional web development frameworks, Streamlit allows developers to create web applications entirely using Python without writing:

- HTML
- CSS
- JavaScript

### Definition

Streamlit is a Python framework that converts Python scripts into interactive web applications in real time.

### Why Streamlit?

Traditional Web Development:

Frontend

├── HTML  
├── CSS  
└── JavaScript

Backend

└── Python

**Streamlit:**

Python Only

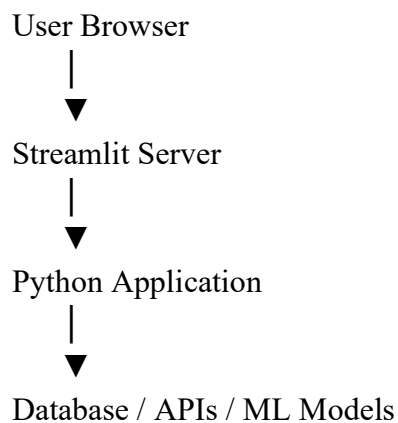


Web Application

## 2. Features of Streamlit

- Pure Python Development
- Fast Application Development
- Interactive Widgets
- Automatic UI Rendering
- Responsive Design
- Data Visualization Support
- Machine Learning Integration
- AI Application Development
- Real-Time Updates
- Deployment Support

## 3. Streamlit Architecture



## 4. Installing Streamlit

### Using PIP

```
pip install streamlit
```

Verify Installation:

```
streamlit --version
```

**Example:**

Streamlit, version 1.45.x

## 5. First Streamlit Application

**Create:**

app.py

**Code:**

```
import streamlit as st

st.title("My First Streamlit App")

st.write("Welcome to Streamlit")
```

**Run:**

streamlit run app.py

**Output:**

Browser Opens Automatically

My First Streamlit App  
Welcome to Streamlit

## 6. Streamlit Execution Model

**Traditional Python:**

Start  
↓  
Execute Once  
↓  
End

**Streamlit:**

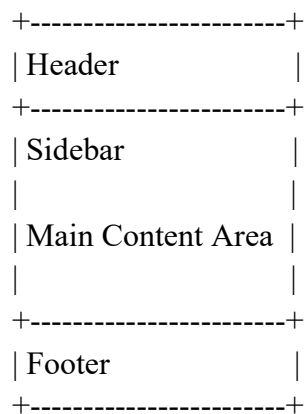
User Action  
|  
▼  
Entire Script Re-runs

**Whenever a user:**

- Clicks a button
- Changes input
- Selects dropdown

The entire script is executed again.

## 7. Streamlit Layout Architecture



## 8. Text Elements

### Title

```
import streamlit as st

st.title("Student Management System")
```

### Header

```
st.header("Student Information")
```

### Subheader

```
st.subheader("Enter Details")
```

### Text

```
st.text("Simple Text")
```

## Markdown

```
st.markdown("""  
# Python  
## Streamlit  
### Dashboard  
""")
```

## Code Display

```
st.code("""  
print("Hello")  
""")
```

# 9. User Input Widgets

## Text Input

```
name = st.text_input(  
    "Enter Name"  
)  
  
st.write(name)
```

## Number Input

```
age = st.number_input(  
    "Enter Age"  
)
```

## Text Area

```
address = st.text_area(  
    "Enter Address"  
)
```

## Password Input

```
password = st.text_input(  
    "Password",  
    type="password"  
)
```

## 10. Selection Widgets

### Select Box

```
course = st.selectbox(  
    "Select Course",  
    ["BTech","MTech","MCA"]  
)
```

### Radio Button

```
gender = st.radio(  
    "Gender",  
    ["Male","Female"]  
)
```

### Checkbox

```
agree = st.checkbox(  
    "I Agree"  
)
```

### Multiselect

```
subjects = st.multiselect(  
    "Select Subjects",  
    ["Python","Java","C++"]  
)
```

## 11. Buttons

```
if st.button("Submit"):  
    st.success("Submitted")
```

## 12. Sliders

```
marks = st.slider(  
    "Marks",  
    0,  
    100  
)
```

```
st.write(marks)
```

## 13. Date and Time Inputs

### Date Picker

```
date = st.date_input(  
    "Select Date"  
)
```

### Time Picker

```
time = st.time_input(  
    "Select Time"  
)
```

## 14. File Upload

```
uploaded_file = st.file_uploader(  
    "Upload File"  
)
```

### Upload CSV Example

```
import pandas as pd  
  
file = st.file_uploader(  
    "Upload CSV",  
    type=["csv"]  
)  
  
if file:  
  
    df = pd.read_csv(file)  
  
    st.dataframe(df)
```

## 15. Displaying Data

### Table

```
st.table(df)
```

## Interactive DataFrame

```
st.dataframe(df)
```

Allows:

- Sorting
- Scrolling
- Filtering

## 16. Images

```
from PIL import Image  
  
img = Image.open("image.jpg")  
  
st.image(img)
```

## 17. Audio

```
st.audio("audio.mp3")
```

## 18. Video

```
st.video("video.mp4")
```

## 19. Metrics

```
st.metric(  
    "Revenue",  
    "$50,000",  
    "$2,000"  
)
```

**Output:**



```
Revenue
$50,000
+ $2,000
```

## 20. Columns

```
col1,col2 = st.columns(2)

with col1:
    st.write("Column 1")

with col2:
    st.write("Column 2")
```

## 21. Sidebar

```
st.sidebar.title("Menu")

choice = st.sidebar.selectbox(
    "Select",
    ["Home","About"]
)
```

**Layout:**

```
Sidebar | Main Page
```

## 22. Containers

```
container = st.container()

with container:
    st.write("Inside Container")
```

## 23. Expanders

```
with st.expander(
    "View Details"
):
    st.write("Hidden Content")
```

## 24. Streamlit Session State

Stores data between reruns.

```
if "count" not in st.session_state:

    st.session_state.count = 0

if st.button("Increment"):

    st.session_state.count += 1

st.write(
    st.session_state.count
)
```

## 25. Charts with Streamlit

### Line Chart

```
import pandas as pd

data = pd.DataFrame({
    "Sales": [10, 20, 30, 40]
})

st.line_chart(data)
```

### Bar Chart

```
st.bar_chart(data)
```

### Area Chart

```
st.area_chart(data)
```

## 26. Using Matplotlib

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
```

```
ax.plot([1,2,3],[10,20,30])  
  
st.pyplot(fig)
```

## 27. Using Seaborn

```
import seaborn as sns  
import matplotlib.pyplot as plt  
  
fig, ax = plt.subplots()  
  
sns.histplot(  
    [1,2,3,4,5],  
    ax=ax  
)  
  
st.pyplot(fig)
```

## 28. Using Plotly

```
import plotly.express as px  
  
fig = px.bar(  
    x=["A","B","C"],  
    y=[10,20,30]  
)  
  
st.plotly_chart(fig)
```

## 29. Streamlit and Pandas

```
import pandas as pd  
  
df = pd.read_csv(  
    "students.csv"  
)  
  
st.dataframe(df)
```

## 30. Streamlit and Databases

### SQLite Example

```
import sqlite3

conn = sqlite3.connect(
    "students.db"
)
```

**Display:**

```
import pandas as pd

df = pd.read_sql(
    "SELECT * FROM students",
    conn
)

st.dataframe(df)
```

## 31. Streamlit and MariaDB

```
import mariadb
import pandas as pd

conn = mariadb.connect(
    host="localhost",
    user="root",
    password="password",
    database="studentdb"
)

query = """
SELECT * FROM students
"""

df = pd.read_sql(
    query,
    conn
)

st.dataframe(df)
```

## 32. Streamlit and Machine Learning

**Example:**

```
from sklearn.linear_model import LinearRegression
```

**Train model:**

```
model.fit(X,y)
```

#### **Prediction:**

```
prediction = model.predict(
    [[hours]]
)

st.write(prediction)
```

## **33. Streamlit and AI**

#### **Applications:**

- Chatbots
- LLM Applications
- AI Assistants
- Recommendation Systems

#### **Example:**

```
prompt = st.text_input(
    "Ask Question"
)

if prompt:
    response = ai_model(prompt)

    st.write(response)
```

## **34. Streamlit Themes**

#### **Theme Configuration**

#### **Create:**

```
streamlit/config.toml
```

#### **Example:**

```
[theme]

primaryColor="#FF4B4B"

backgroundColor="#FFFFFF"
```

```
secondaryBackgroundColor="#F0F2F6"

textColor="#31333F"

font="sans serif"
```

## 35. Dark Theme

```
[theme]

primaryColor="#00BFFF"

backgroundColor="#0E1117"

secondaryBackgroundColor="#262730"

textColor="#FFFFFF"
```

## 36. Responsive Web Interface

**Streamlit automatically supports:**

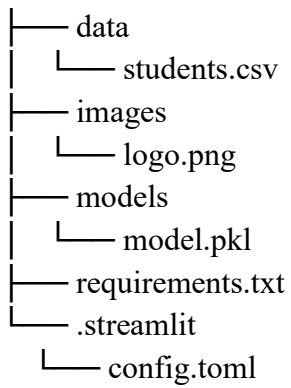
- Desktop
- Laptop
- Tablet
- Mobile

**Architecture:**

```
Browser
|
Responsive Layout
|
Streamlit Components
```

## 37. Streamlit Project Structure

```
Project
|
├── app.py
```



## 38. Deployment

### Deploy on:

- Streamlit Community Cloud
- Docker
- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform

## 39. Streamlit + Docker

### Dockerfile:

```
FROM python:3.11

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

EXPOSE 8501

CMD [
    "streamlit",
    "run",
    "app.py",
    "--server.address=0.0.0.0"
]
```

### Run:

```
docker build -t streamlit-app .
```

```
docker run -p 8501:8501 streamlit-app
```

## 40. Complete Example: Student Result Dashboard

```
import streamlit as st
import pandas as pd

st.title(
    "Student Result Dashboard"
)

file = st.file_uploader(
    "Upload CSV"
)

if file:

    df = pd.read_csv(file)

    st.dataframe(df)

    st.write(
        "Maximum Marks",
        df["Total"].max()
    )

    st.write(
        "Minimum Marks",
        df["Total"].min()
    )

    st.bar_chart(df["Total"])
```

### Advantages of Streamlit

1. Pure Python development.
2. No HTML/CSS/JavaScript required.
3. Rapid application development.
4. Excellent for Data Science.
5. Excellent for AI and ML.
6. Built-in responsiveness.



7. Easy deployment.

### **Limitations of Streamlit**

1. Less suitable for highly customized enterprise frontends.
2. Entire script reruns on interaction.
3. Limited compared to full frameworks like Django for large-scale web applications.

### **Summary**

**Streamlit** is a modern Python framework for building responsive web applications, dashboards, data science tools, machine learning interfaces, and AI applications entirely in Python. It provides built-in UI components, charts, data visualization, database connectivity, file handling, session management, and deployment capabilities. By eliminating the need for HTML, CSS, and JavaScript, Streamlit enables rapid development of professional, interactive, and responsive applications, making it one of the most popular frameworks for Data Science, Machine Learning, Artificial Intelligence, Cyber Security Analytics, and educational projects.

## **Exercise:01:**

### **Streamlit + Pandas + CSV application:**

#### **Program: Student Marks Calculator using Streamlit**

```
import streamlit as st
import pandas as pd
import os

st.title("Student Marks Management System")

st.header("Enter Student Details")

# Student Information
student_name = st.text_input("Enter Student Name")

# Subject Marks
english = st.number_input("English Marks", min_value=0, max_value=100)
hindi = st.number_input("Hindi Marks", min_value=0, max_value=100)
maths = st.number_input("Maths Marks", min_value=0, max_value=100)
sst = st.number_input("SST Marks", min_value=0, max_value=100)
science = st.number_input("Science Marks", min_value=0, max_value=100)

if st.button("Calculate and Save"):
```

```

total = english + hindi + maths + sst + science
percentage = total / 5

st.success("Result Calculated Successfully")

st.write("### Result")
st.write(f"**Total Marks:** {total}")
st.write(f"**Percentage:** {percentage:.2f}%")

# Create DataFrame
data = {
    "Student Name": [student_name],
    "English": [english],
    "Hindi": [hindi],
    "Maths": [maths],
    "SST": [sst],
    "Science": [science],
    "Total": [total],
    "Percentage": [percentage]
}

df = pd.DataFrame(data)

filename = "student_marks.csv"

# Append data if file exists
if os.path.exists(filename):
    df.to_csv(filename,
              mode='a',
              header=False,
              index=False)
else:
    df.to_csv(filename,
              index=False)

st.success("Data Saved to student_marks.csv")

st.dataframe(df)

# Display all saved records
if st.button("View All Records"):

    try:
        records = pd.read_csv("student_marks.csv")

        st.subheader("All Student Records")

        st.dataframe(records)

```

```
except FileNotFoundError:  
    st.warning("No records found.")
```

## Install Required Packages

```
pip install streamlit pandas
```

## Run the Application

```
streamlit run app.py
```

## Sample CSV Output

```
Student Name,English,Hindi,Maths,SST,Science,Total,Percentage  
John,85,78,92,88,90,433,86.6  
Alice,75,80,70,85,78,388,77.6
```

## Features Covered

- Streamlit Web Interface
- User Input Forms
- Subject-wise Marks Entry
- Total Calculation
- Percentage Calculation
- Pandas DataFrame Creation
- CSV File Storage
- Append New Records to Existing CSV
- Display Saved Records
- Responsive Web Interface

## Exercise: 02:

## Streamlit + Pandas + MariaDB application:

### Features

- ✓ Accept Student Name and Marks
- ✓ Calculate Total
- ✓ Calculate Percentage
- ✓ Save Record to CSV File
- ✓ Save Record to MariaDB Database
- ✓ Display Result
- ✓ View All Records from Database

## Step 1: Install Required Packages

```
pip install streamlit pandas mariadb
```

## Step 2: Create MariaDB Database

### Login to MariaDB:

```
mariadb -u root -p
```

### If MariaDB is Running Inside Docker

#### For example:

```
docker run -d \  
-p 3306:3306 \  
--name mariadb \  
-e MARIADB_ROOT_PASSWORD=root123 \  
mariadb
```

#### Use:

```
conn = mariadb.connect(  
    host="localhost",  
    port=3306,  
    user="root",  
    password="root123",  
    database="studentdb"  
)
```

If Streamlit is running in another Docker container on the same Docker network, use the container name instead of localhost:

```
host="mariadb"
```

### Create database:

```
CREATE DATABASE studentdb;

USE studentdb;

CREATE TABLE student_marks (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_name VARCHAR(100),
    english INT,
    hindi INT,
    maths INT,
    sst INT,
    science INT,
    total INT,
    percentage DECIMAL(5,2)
);
```

### Step 3: Streamlit Program

```
import streamlit as st
import pandas as pd
import mariadb
import os

# -----
# MariaDB Connection Function
# -----

def get_connection():

    conn = mariadb.connect(
        host="localhost",
        port=3306,
        user="root",
        password="your_password",
        database="studentdb"
    )

    return conn

# -----
# Streamlit UI
# -----

st.title("Student Marks Management System")

st.header("Enter Student Details")

student_name = st.text_input(
```

```

    "Student Name"
)

english = st.number_input(
    "English",
    min_value=0,
    max_value=100
)

hindi = st.number_input(
    "Hindi",
    min_value=0,
    max_value=100
)

maths = st.number_input(
    "Maths",
    min_value=0,
    max_value=100
)

sst = st.number_input(
    "SST",
    min_value=0,
    max_value=100
)

science = st.number_input(
    "Science",
    min_value=0,
    max_value=100
)

# -----
# Save Data
# -----

if st.button("Calculate and Save"):

    total = (
        english +
        hindi +
        maths +
        sst +
        science
    )

    percentage = total / 5

    st.success("Result Calculated")

```

```

st.write("### Result")

st.write(
    f"Total Marks : {total}"
)

st.write(
    f"Percentage : {percentage:.2f}%"
)

# -----
# Save to CSV
# -----

student_data = {

    "Student Name": [student_name],
    "English": [english],
    "Hindi": [hindi],
    "Maths": [maths],
    "SST": [sst],
    "Science": [science],
    "Total": [total],
    "Percentage": [percentage]

}

df = pd.DataFrame(student_data)

csv_file = "student_marks.csv"

if os.path.exists(csv_file):

    df.to_csv(
        csv_file,
        mode="a",
        header=False,
        index=False
    )

else:

    df.to_csv(
        csv_file,
        index=False
    )

st.success(
    "Data saved to CSV file"
)

```

```

# -----
# Save to MariaDB
# -----

try:

    conn = get_connection()

    cursor = conn.cursor()

    query = """
INSERT INTO student_marks
(
    student_name,
    english,
    hindi,
    maths,
    sst,
    science,
    total,
    percentage
)
VALUES
(
    ?, ?, ?, ?, ?, ?, ?, ?
)
"""

    cursor.execute(

        query,

        (
            student_name,
            english,
            hindi,
            maths,
            sst,
            science,
            total,
            percentage
        )
    )

    conn.commit()

    st.success(
        "Data saved to MariaDB"
    )

```



```

        cursor.close()
        conn.close()

except mariadb.Error as e:

    st.error(
        f"Database Error: {e}"
    )

st.dataframe(df)

# -----
# View All Records From Database
# -----

if st.button("View All Records"):

    try:

        conn = get_connection()

        query = """
        SELECT *
        FROM student_marks
        """

        records = pd.read_sql(
            query,
            conn
        )

        st.subheader(
            "Student Records"
        )

        st.dataframe(records)

        conn.close()

    except mariadb.Error as e:

        st.error(
            f"Database Error: {e}"
        )

```

### Expected Database Output

```
SELECT * FROM student_marks;
```

| id | student_name | english | hindi | maths | sst | science | total | percentage |
|----|--------------|---------|-------|-------|-----|---------|-------|------------|
| 1  | John         | 85      | 78    | 92    | 88  | 90      | 433   | 86.60      |
| 2  | Alice        | 75      | 80    | 70    | 85  | 78      | 388   | 77.60      |

This project demonstrates:

- Streamlit Forms
- Pandas DataFrames
- CSV File Handling
- MariaDB Connectivity
- SQL INSERT Operations
- SQL SELECT Operations
- Database Transactions
- Responsive Web Interface Development with Python.